

AgentCore Privilege Escalation

Execution-role abuse through a code interpreter

Technical Documentation

Ritvik Indupuri

1 Project overview

This project documents a controlled AWS lab in which a caller uses an Amazon Bedrock AgentCore Code Interpreter to execute AWS CLI commands under an attached IAM execution role. The role grants S3 permissions, allowing the interpreter session to enumerate a customer-data bucket and its object keys.

The security issue is the combination of access to a programmable execution tool and permissions delegated to that tool. An operator who can create or invoke a suitably privileged interpreter can cause AWS requests to run as its execution role. The original caller's identity and the runtime's assumed-role identity are separate authorization contexts.

The repository contains a short Boto3 script, an S3 policy excerpt, and screenshots of role inspection, interpreter creation, command execution, and S3 enumeration. Its implemented workflow is deliberately small: one session, four commands, and streamed terminal output.

Demonstrated outcome

Evidence	Supported conclusion
STS output	Commands execute as an assumed-role session for AgentCodeInterpreterRole.
Account bucket listing	The execution context can enumerate the returned S3 bucket.
Bucket and prefix listings	The session lists customer_data/ and three object names with sizes.
Role policy	The supplied policy grants bucket listing and GetObject across that bucket's object namespace.

The README describes the starting caller as lacking direct S3 access. The supplied screenshots establish the runtime identity and successful listings, but do not show a separate denied S3 request from the original caller. Object downloads and file contents are also not shown. The demonstrated result is role-backed enumeration; object retrieval is a potential consequence of the policy grant.

Scope and terminology

No language model is invoked by privesc_demo.py. The script uses the supported executeCommand operation directly, so this is an IAM delegation and tool-access case rather than a prompt-injection demonstration. AWS documents direct Code Interpreter use without an agent framework. [A1]

Sections 2–3 explain the system and execution flow; Sections 4–5 document the script, policy, and evidence; Sections 6–8 cover mitigations, operation, and references. Live AWS execution was not repeated for this document.

2 System architecture

The operator launches the Python script from a terminal with AWS credentials configured. AgentCore authorizes the caller's session and invocation requests; commands inside the interpreter use its execution-role context.

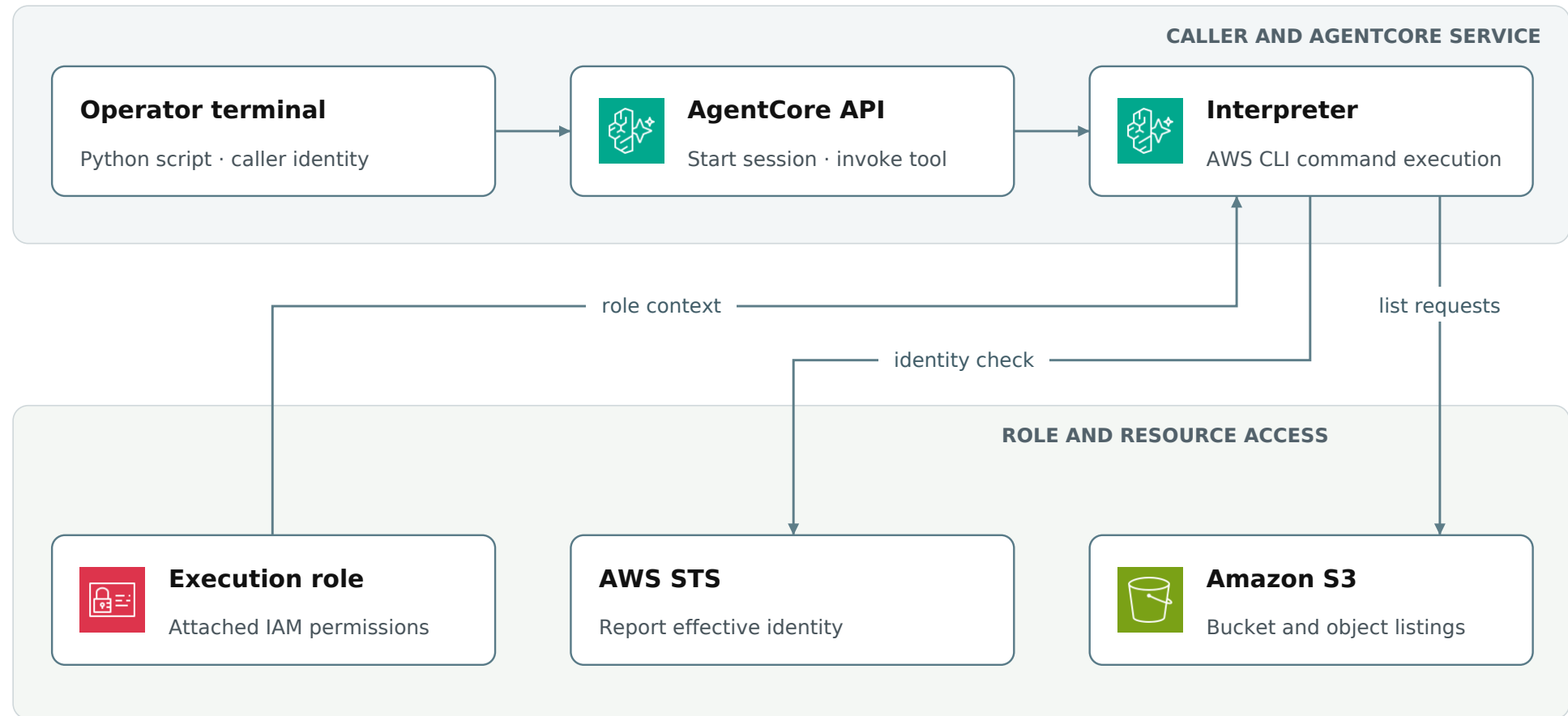


Figure 1. Caller requests, execution-role context, and the two AWS services queried by the lab.

The IAM role is a permission source, not a network hop. STS reports the active identity; S3 evaluates the requests made from that runtime. Command results return through the invocation stream to the terminal. Official AgentCore icons represent the execution service; no LLM or autonomous agent participates in this path.

3 Interpreter request lifecycle

Preparation establishes the custom interpreter and its role. The checked-in script then starts one session and submits four commands sequentially, consuming each response stream before the next command.

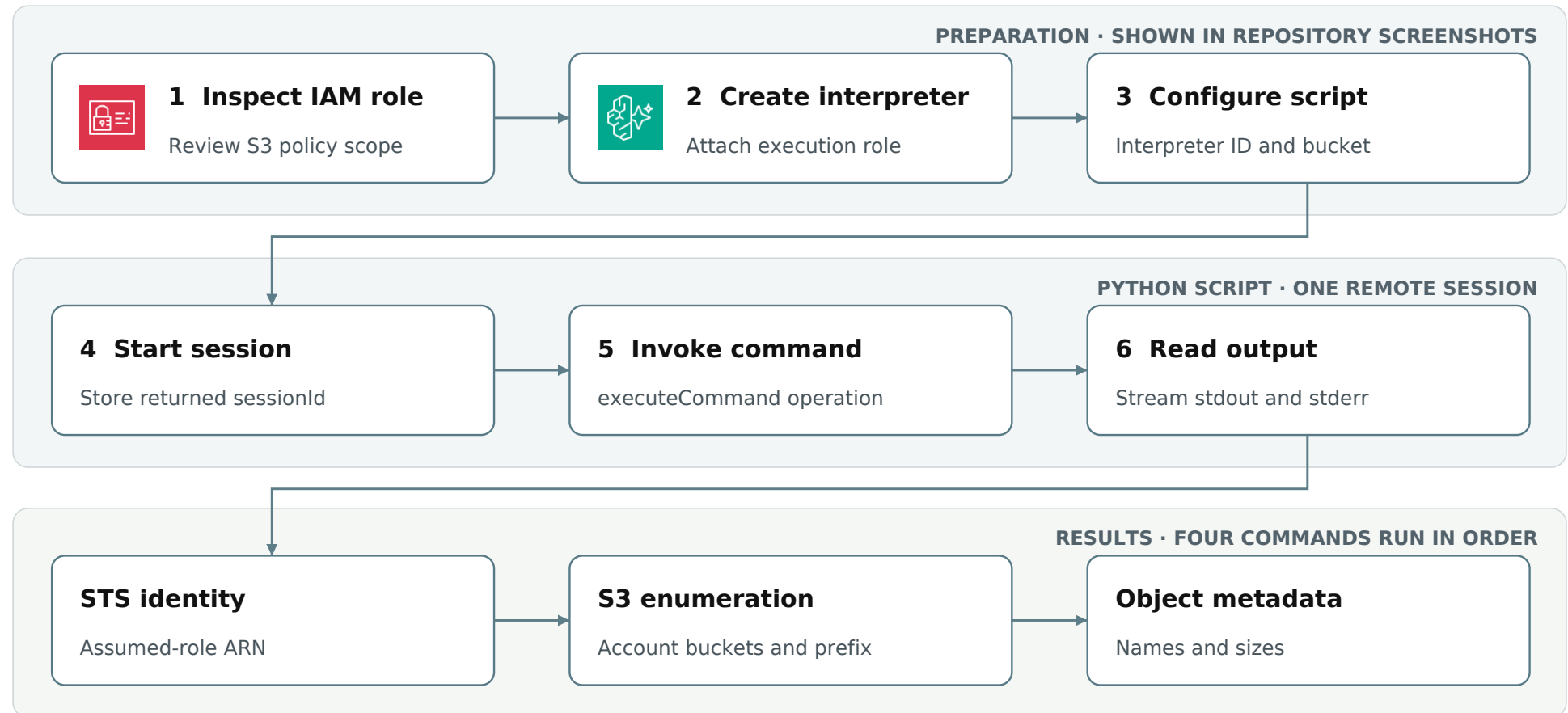


Figure 2. Preparation, session invocation, and evidence collection. The bottom row groups the four command results.

The commands check STS identity, list account buckets, list the selected bucket, and list its `customer_data/` prefix. Steps 5–6 repeat for each command. The script relies on an existing interpreter and does not include creation, a model decision loop, or automatic session cleanup.

4 Implementation and IAM policy

Python execution path

`privesc_demo.py` imports Boto3 and creates a `bedrock-agentcore` client using the SDK's configured credentials and region. `CODE_INTERPRETER_ID` identifies a pre-created custom tool.

`start_code_interpreter_session` returns the `sessionId` reused by every command.

The helper `run_command_and_print_results` sends the interpreter identifier, session identifier, `name="executeCommand"`, and `arguments={"command": command}`. It walks `response["stream"]`, reads `result.structuredContent`, and prints nonempty `stdout` and `stderr` fields. This matches the command-based API operation; the checked-in file does not use `os.popen` or ask a model to generate Python. [A4]

Order	Remote command and purpose
1	<code>aws sts get-caller-identity</code> : identify the runtime principal.
2	<code>aws s3 ls</code> : enumerate account buckets.
3	<code>aws s3 ls s3://<lab-bucket>/</code> : list the selected bucket.
4	<code>aws s3 ls s3://<lab-bucket>/customer_data/</code> : list the data prefix.

Each command begins with `PAGER=cat`. The script performs listings; it contains no `aws s3 cp`, `sync`, object-body retrieval, credential extraction, or recursive `--recursive` listing. Its output is printed to the terminal rather than written to a report store.

Role permissions in `policy-excerpt.json`

Permission	Resource scope	Effect
<code>s3:ListAllMyBuckets</code>	*	Lists account bucket names.
<code>s3:ListBucket</code>	Lab bucket ARN	Lists keys and prefixes in that bucket.
<code>s3:GetObject</code>	Lab bucket ARN followed by <code>/*</code>	Grants object retrieval throughout the bucket.

The policy scopes object access to one bucket but not to a narrower object prefix. Its account identifier is a placeholder. It is an identity-policy excerpt, not the caller's complete IAM policy, the role's trust policy, or a full effective-permissions evaluation. Other applicable policies and denies can affect access.

Provisioning evidence

The creation screenshot uses the `bedrock-agentcore-control` CLI in `us-east-1`, specifies the execution-role ARN, and chooses `networkMode PUBLIC`. The returned custom interpreter reports `READY`. `PUBLIC` describes outbound internet connectivity; it does not remove IAM authorization from direct invocation. AWS also documents S3 access in `SANDBOX` mode. [A2] [A5]

5 Evidence and observed results

The following observations come from the repository’s terminal screenshots and policy artifact. Filenames are evidence references, not additional tests performed during preparation of this document.

Repository evidence	Observation
Role Enumeration.png	ListRoles returns AgentCodeInterpreterRole and its ARN.
Overprivileged IAM Policy.png	The role policy shows account bucket enumeration, bucket listing and object read permissions.
Create the custom code interpreter.png	Creation supplies PUBLIC mode and the execution role; the response returns an interpreter identifier and READY status.
Code interpreter exploit script.png	The script starts a session and invokes executeCommand, then prints structured output.
Privilege Boundary Proof.png	STS reports an assumed-role identity for AgentCodeInterpreterRole. This image is runtime identity evidence.
S3 access gained.png	The interpreter-backed run returns an S3 bucket name.
List bucket contents.png	The output contains customer_data/ as a listed prefix.
Sensitive file listing.png	The final listing returns customer_profiles.csv, financial_transactions.csv and support_tickets.json.

Object metadata returned

Object name	Size shown
customer_profiles.csv	522 bytes
financial_transactions.csv	386 bytes
support_tickets.json	764 bytes

These names and sizes are listing metadata. They do not establish the content of the objects or prove that file bodies were downloaded. The README states that the demonstration used a controlled lab and did not compromise actual customer data.

How to interpret the evidence

Some terminal captures retain a NoSuchBucket error from a preceding request alongside later successful listings. The supported conclusions above rely on the specific successful output shown. The screenshots use multiple session identifiers, so they should be read as a progression of lab runs rather than a single uninterrupted execution log.

The clearest demonstrated boundary is the runtime’s assumed-role execution context. The README’s comparison with a less-privileged caller remains the stated lab scenario; a complete before-and-after proof would also preserve the caller identity and its denied request to the same target.

6 Security analysis and mitigations

Authorization boundary

Two permission sets matter: the caller's ability to use the interpreter, and the execution role's ability to access data. When the role can perform actions beyond the intended authority of the caller, direct command execution provides a route to those actions. AWS still evaluates the runtime requests under the delegated identity; this demonstration does not establish an AWS IAM enforcement bypass.

For the creation path, control access to interpreter creation and the ability to pass the selected role. For an existing interpreter, control session-start and invocation access as well. Restricting creation alone leaves access to already provisioned tools as a separate concern. The repository does not include a complete caller policy or role trust policy.

Recommended controls

Control	Purpose
Scope the execution role	Grant only required actions, buckets and prefixes. Separate artifact-processing data from unrelated sensitive data.
Constrain role passing	Allow iam:PassRole only for approved execution-role ARNs; use the intended service condition where appropriate.
Restrict tool use	Limit who can create interpreters, start sessions and invoke commands. Review access to existing tools.
Review role trust	Permit the intended AWS service and constrain the account context as appropriate for the deployment.
Reduce network access	Choose the network mode and endpoints required by the workload. Network restrictions complement IAM controls.

AWS explicitly warns that permission to pass a stronger role can let a service perform actions the caller cannot perform directly. Scope the PassRole resource set and service target accordingly. These are recommended controls; the repository does not show a deployed remediation or a post-hardening test. [A3]

Monitoring design

Correlate interpreter creation and role selection with subsequent session activity and AWS requests from the execution role. Retain the caller, interpreter identifier, role ARN, session identifier, region and target resources. Verify that the required event categories are actually collected before relying on a CloudTrail trail for detection.

PassRole is a permission check, not a standalone API event. Review the service's create or update event to determine which role was assigned. Direct interpreter calls can be legitimate, so alerts should consider unusual callers, privileged roles and unexpected data access rather than flagging every direct invocation. [A3]

7 Lab operation and next steps

Required environment

Use an authorized AWS lab account, a custom Code Interpreter with its execution role already attached, and a test S3 bucket. The local runner needs Python and Boto3 with AgentCore client support. The remote command environment needs the AWS CLI, as demonstrated in the screenshots. The creation example uses us-east-1; configure the runner for the interpreter's region.

Local preparation

```
git clone https://github.com/ritvikindupuri/AgentCore-Privilege-Escalation.git
cd AgentCore-Privilege-Escalation
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install boto3
```

Configure the intended AWS profile and region through the standard SDK configuration. Replace `CODE_INTERPRETER_ID` and both bucket paths in `privesc_demo.py` with the lab resources. The script does not accept these values as command-line arguments or read `policy-excerpt.json`.

```
python3 privesc_demo.py
```

The run starts a remote session and performs the four configured read-oriented commands. Evaluate STS identity and each command's output independently. Printed stderr is not converted into a structured test result, and the helper does not implement an explicit stop-on-command-failure decision.

Session lifecycle

The checked-in script does not explicitly stop its session or delete the custom interpreter. Operational cleanup should stop the session and remove temporary resources when no longer needed. AWS's direct-use examples show explicit stop calls, including cleanup in a finally block. [A1]

Next steps

Parameterize the interpreter, region and bucket; validate stream event shapes; retain session identifiers and exit results; and add cleanup in a finally block. Pin tested dependency versions for repeatable lab execution.

Extend the evidence with a paired caller-versus-runtime test using the same target. Preserve the caller's effective permissions and role trust policy, then apply least-privilege changes and repeat the test to demonstrate the prevention control. These additions would strengthen the validation without requiring access to real sensitive data.

8 Implementation references

Repository artifacts

These references identify the exact revision reviewed for this document. The implementation review covered the Python script, policy excerpt, README and all supplied PNG evidence.

Artifact	Reference
Python workflow	privesc_demo.py
S3 permissions	policy-excerpt.json
Lab narrative	README.md
Terminal evidence	.assets directory

Repository revision: 2a338faae44f436e3a9b59d6041ef8ca0724acf2

AWS documentation

ID	Reference and relevance
A1	Using AgentCore Code Interpreter directly
A2	Terminal commands with an execution role
A3	Passing IAM roles to AWS services
A4	Boto3 InvokeCodeInterpreter reference
A5	Creating an AgentCore Code Interpreter

The AWS references support API behavior, role delegation and network-mode explanations. The repository supplies the project-specific evidence. The service icon artwork is from the official AWS Architecture Icons collection.

Validation scope

The diagrams follow the checked-in request sequence and distinguish the local caller from the remote execution identity. No LLM component, autonomous agent, external data-transfer destination, or demonstrated file download has been added to the flow.

Code and screenshot inspection support the described implementation and observed listings. The lab was not recreated during documentation preparation, and the recommended mitigations were not applied or tested as part of this review.